Optimisation

Programmation linéaire

CentraleSupélec - Cursus ingénieur

1. Programmation linéaire continue
2. Algorithme du simplexe
3. Dualité
4. Programmation linéaire discrète
   - Problèmes de flot
   - Problèmes entiers célèbres

Plan

1. Programmation linéaire continue
2. Algorithme du simplexe
3. Dualité
4. Programmation linéaire discrète
   - Problèmes de flot
   - Problèmes entiers célèbres

Ecriture formelle

On veut résoudre

$$\inf_{\substack{Ax=b \\ x\geqslant 0}} c\cdot x \qquad \text{où} \quad x\in\mathbb{R}^n$$

où $A\in\mathcal{M}_{m\times n}$ avec $m\leqslant n$ et $\mathrm{rg}(A)=m$.

On note $X_{ad}=\{x\in\mathbb{R}^n_+, \quad Ax=b\}$. C'est un polyèdre.
Que sont les faces, arrêtes, etc. ?

Définition

1. *Un point $x\in X_{ad}$ est dit solution* **admissible**
2. *Un sommet de X_{ad} est appelé solution* **extrémale**.

Lemme

1. *Le problème admet au moins une solution si l'infimum est minoré inférieurement*
2. *Pour une solution extrémale* x *il existe une permutation des colonnes de* A *telle que* $\mathrm{A} = [\mathrm{B}\ \mathrm{N}]$ *où* B *est inversible et dans cette base* $\mathrm{x} = (\mathrm{x}_\mathrm{B}, \mathrm{x}_\mathrm{N})$ *où* $\mathrm{x}_\mathrm{N} = 0$ *et* $\mathrm{x}_\mathrm{B} = \mathrm{B}^{-1}\mathrm{b} \geqslant 0$. *On dit que* x *est une solution* **basique**.
3. *S'il existe une solution optimale au programme linéaire alors il existe une solution optimale basique.*

Idées :

1. On utilise que l'ensemble admissible est un cône fermé et que $\mathrm{x} \longrightarrow \mathrm{c} \cdot \mathrm{x}$ est continue.
2. On utilise qu'un point extrémal n'est pas combinaison convexe non triviale de points de X_{ad}.

Proposition

Soit $\bar{x}$ une solution basique. Alors pour tout x admissible

$$c \cdot (x - \bar{x}) = \underbrace{(c_N - {}^t(B^{-1}N)c_B)}_{\tilde{c}_N} \cdot x_N$$

$\bar{x}$ est donc une solution optimale si et seulement si $\tilde{c}_N \geqslant 0$.

Proposition

Soit $\bar{x}$ une solution basique. Alors pour tout x admissible

$$c \cdot (x - \bar{x}) = \underbrace{(c_N - {}^t(B^{-1}N)c_B)}_{\tilde{c}_N} \cdot x_N$$

$\bar{x}$ est donc une solution optimale si et seulement si $\tilde{c}_N \geqslant 0$.

Remarque (essentielle)

Soit x basique. Si $\tilde{c}_N \cdot e_i < 0$ alors

$$x(\varepsilon) = (B^{-1}(b - \varepsilon N_i), \varepsilon e_i) \quad \textit{satisfait} \quad Ax(\varepsilon) = b$$

et $c \cdot (x(\varepsilon) - x) = \varepsilon \tilde{c}_N \cdot e_i < 0$ pour $\varepsilon > 0$. Donc le coût décroît

Plan

1. Programmation linéaire continue
2. Algorithme du simplexe
3. Dualité
4. Programmation linéaire discrète
   - Problèmes de flot
   - Problèmes entiers célèbres

Algorithme du simplexe

On veut résoudre

$$\min_{\substack{Ax=b \\ x \geqslant 0}} c \cdot x \quad x \in \mathbb{R}^n \quad A \in \mathcal{M}_{m \times n}$$

Algorithme du simplexe :

1) Soit x_k un point admissible basique :

$$A_k = [B_k \; N_k], \quad c_k = (c_{k,B}, c_{k,N})$$

Calcul du coût réduit

$$\tilde{c}_k = c_{k,N} - {}^t N_k \, {}^t B_k^{-1} c_{k,B}$$

- Si $\tilde{c}_k \geqslant 0$ alors x_k est une solution basique optimale.
- Sinon soit i l'indice de la composante la plus négative de $\tilde{c}_k$

2) Notons a_i la colonne de A_k (en fait N_k) correspondante et posons

$$x_k(\varepsilon) = (x_{k,B}(\varepsilon), x_{k,N}(\varepsilon))$$
$$x_{k,N}(\varepsilon) = \varepsilon e_i, \quad x_{k,B}(\varepsilon) = B_k^{-1}(b - \varepsilon a_i)$$

3) Soit on peut choisir $\varepsilon > 0$ aussi grand que l'on veut et alors le problème n'est pas borné inférieurement,

4) Soit il existe ε_k maximal tel que $x_k(\varepsilon_k)$ soit admissible mais pas $x_k(\varepsilon)$ pour $\varepsilon > \varepsilon_k$. Ceci signifie qu'une composante $j \neq i$ de $x_{k,B}(\varepsilon_k)$ est nulle. On pose $x_{k+1} = x_k(\varepsilon_k)$ et on réordonne la base en échangeant les colonnes i et j de A.

Initialisation :

• Utiliser l'agorithme du simplexe pour le problème

$$\min_{\substack{\tilde{A}x+y=\tilde{b} \\ (x,y)\geqslant 0}} 1 \cdot y \qquad (x,y) \in \mathbb{R}^{n+m}$$

où $\tilde{b}_i = |b_i|$ et $\tilde{A}$ correspondant.

• Initialiser avec $(y_0, x_0) = (\tilde{b}, 0)$ qui est un vecteur admissible basique.

Plan

1. Programmation linéaire continue
2. Algorithme du simplexe
3. Dualité
4. Programmation linéaire discrète
   - Problèmes de flot
   - Problèmes entiers célèbres

Problème primal :

$$\min_{\substack{Ax=b \\ x \geqslant 0}} c \cdot x \quad x \in \mathbb{R}^n \quad A \in \mathcal{M}_{m \times n}$$

Lagrangien

$$L(x, p) = c \cdot x + p \cdot (b - Ax)$$

Coût dual :

$$G(p) = \min_{x \geqslant 0} L(x, p) = \begin{cases} p \cdot b & \text{si } c - {}^t\!Ap \geqslant 0 \\ -\infty & \text{sinon} \end{cases}$$

Problème dual :

$$\sup_{{}^{t}Ap \leqslant c} b \cdot p \quad p \in \mathbb{R}^m.$$

Théorème

Si le problème primal ou dual a une solution finie alors l'autre aussi. De plus il existe un point selle $(\bar{x}, \bar{p})$ du Lagrangien qui est admissible pour chacun des problèmes et tel que

$$\min_{\substack{Ax=b \\ x \geqslant 0}} c \cdot x = c \cdot \bar{x} = b \cdot \bar{p} = \sup_{{}^{t}Ap \leqslant c} b \cdot p.$$

Le point vérifie également la condition d'écarts complémentaires $\bar{x} \cdot (c - {}^{t}A\bar{p}) = 0.$

Exemple

Soit un centre d'appels téléphoniques de paramètres

- c_t, *le nombre de clients devant être servis à l'instant* t *où* $t \in [0, T]$
- k *tranches horaires de travail* $[\alpha_i, \beta_i]$ *pour* $1 \leqslant i \leqslant k$ *avec* $0 \leqslant \alpha_i < \beta_i \leqslant T$.
- S_i, *le salaire perçu sur la tranche* i.

On veut minimiser le coût total sous la contrainte de service satisfait.

1. *Introduire des variables et exprimer les contraintes et le coût comme un programme linéaire.*
2. *Soient* $0 = \alpha_1 < \alpha_2 < \beta_1 < \alpha_3 < \beta_2 < \beta_3 = 24$, $S = (4, 5, 6)$ *et* $c = (5, 10, 6, 12, 4)$ *la suite du nombre de clients sur chaque intervalle à nombre d'opérateurs constant. Résoudre le problème dual.*

Plan

1 Programmation linéaire continue

2 Algorithme du simplexe

3 Dualité

4 Programmation linéaire discrète
- Problèmes de flot
- Problèmes entiers célèbres

On veut résoudre pour P polyèdre :

$$\min_{x \in P \cap \mathbb{Z}^n} c \cdot x$$

Quand a t-on :

$$\min_{x \in P \cap \mathbb{Z}^n} c \cdot x = \min_{x \in P} c \cdot x \ ?$$

La première remarque qui peut sauter aux yeux est d'imaginer que résoudre un (PMD) revient à "arrondir" la solution de sa relaxation continue. L'exemple suivant témoigne de l'insuffisance de cette remarque :

Prenons un PMD à deux variables et une seule contrainte où toutes les fonctions sont linéaires, ce qui constitue le cas le plus simple que l'on puisse imaginer.

$$\begin{aligned} &\text{Maximiser } 10x_1 + 11x_2 \\ &10x_1 + 12x_2 \leq 59 \\ &x_1 \text{ et } x_2 \geq 0 \\ &x_1, x_2 \text{ entiers.} \end{aligned}$$

En dessinant le domaine de définition, on obtient la figure 1.1 suivante : On remarque

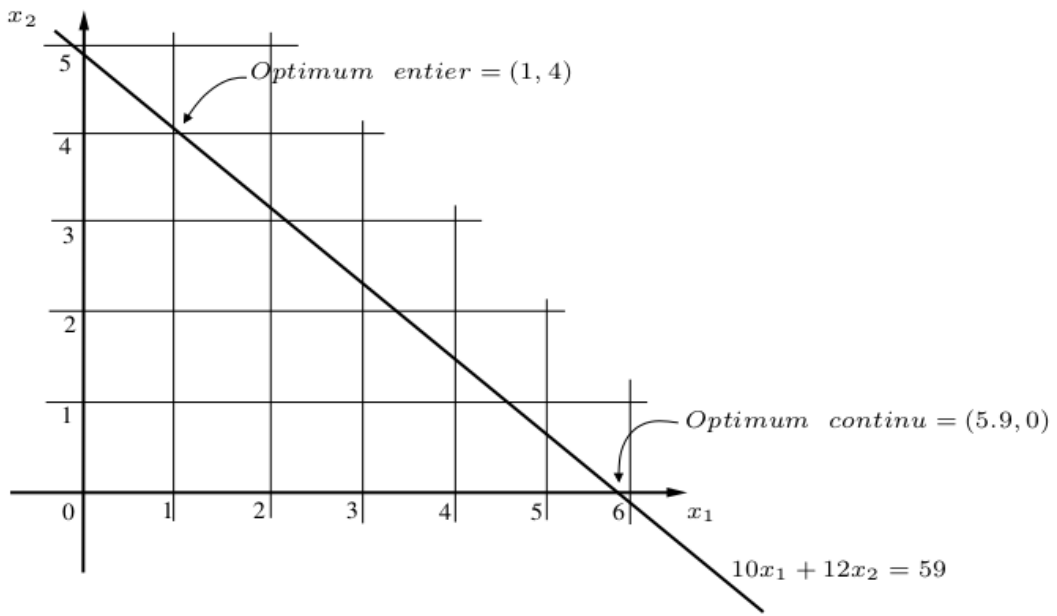


FIGURE 1.1 Ecart entre optimum entier et continu

P Polyèdre : il existe $A \in \mathcal{M}_{m \times n}$ et $b \in \mathbb{R}^m$ tels que

$$P = \{x, Ax \leqslant b\}$$

P Polyèdre : il existe $A \in \mathcal{M}_{m \times n}$ et $b \in \mathbb{R}^m$ tels que

$$P = \{x, Ax \leqslant b\}$$

Définition

L'enveloppe convexe d'un ensemble est le plus petit ensemble convexe le contenant.

Lemme

- *P est l'enveloppe convexe de ses sommets S (points basiques).*
- *Pour* J *convexe* $\min_P J = \min_S J$.

Lemme

- *Si* $A \in \mathcal{M}_n(\mathbb{Z})$ *est inversible alors on a l'équivalence*

$$\forall b \in \mathbb{Z}^n \quad A^{-1}b \in \mathbb{Z}^n \Leftrightarrow \det(A) = \pm 1$$

- *Dans ce cas et si* $b \in \mathbb{Z}^n$ P *est l'enveloppe convexe de* $P \cap \mathbb{Z}^n$.

Lemme

- *Si* $A \in \mathcal{M}_n(\mathbb{Z})$ *est inversible alors on a l'équivalence*

$$\forall b \in \mathbb{Z}^n \quad A^{-1}b \in \mathbb{Z}^n \Leftrightarrow \det(A) = \pm 1$$

- *Dans ce cas et si* $b \in \mathbb{Z}^n$ P *est l'enveloppe convexe de* $P \cap \mathbb{Z}^n$.

Définition

- *Une telle matrice est dite* **unimodulaire**
- *Si* $m \neq n$ *la matrice* A *est dite* **totalement unimodulaire** *si toute sous-matrice carrée est unimodulaire ou non inversible.*

Lemme

Si A *est à coeff* ± 1 *ou* 0 *avec au plus un* 1 *par colonne et un* -1 *par colonne alors* A *est totalement unimodulaire.*

Lemme

Si A est à coeff ± 1 ou 0 avec au plus un 1 par colonne et un -1 par colonne alors A est totalement unimodulaire.

Corollaire

Soit A totalement unimodulaire et $b \in \mathbb{Z}^n$ (et $c \in \mathbb{R}^n$). Alors les solutions basiques du problème $\min_{x \in P} c \cdot x$ sont entières.

En particulier si le problème possède un minimum alors il existe une solution basique entière qui réalise ce minimum.

Lemme

Si A est à coeff ± 1 ou 0 avec au plus un 1 par colonne et un -1 par colonne alors A est totalement unimodulaire.

Corollaire

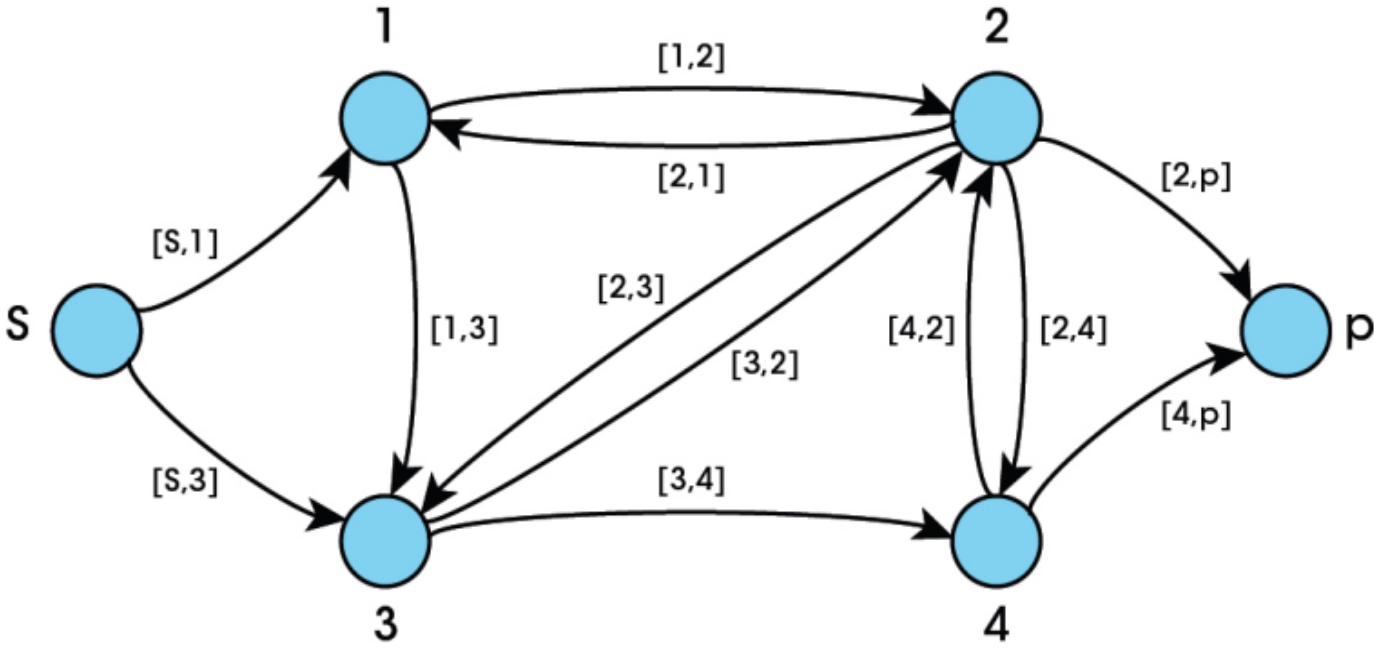
Soit A totalement unimodulaire et $b \in \mathbb{Z}^n$ (et $c \in \mathbb{R}^n$). Alors les solutions basiques du problème $\min_{x \in P} c \cdot x$ sont entières.

En particulier si le problème possède un minimum alors il existe une solution basique entière qui réalise ce minimum.

Exemple : Centre d'appels. T = nb. de fractions de tranche horaires à effectif constant. Soit U la matrice de coefficient $u_{ij} = 1$ si la fraction de tranche horaire j est dans la tranche horaire i. La contrainte s'écrit $Ux \geqslant c$ dans $\mathbb{R}^T$. U est totalement unimodulaire.

Plan

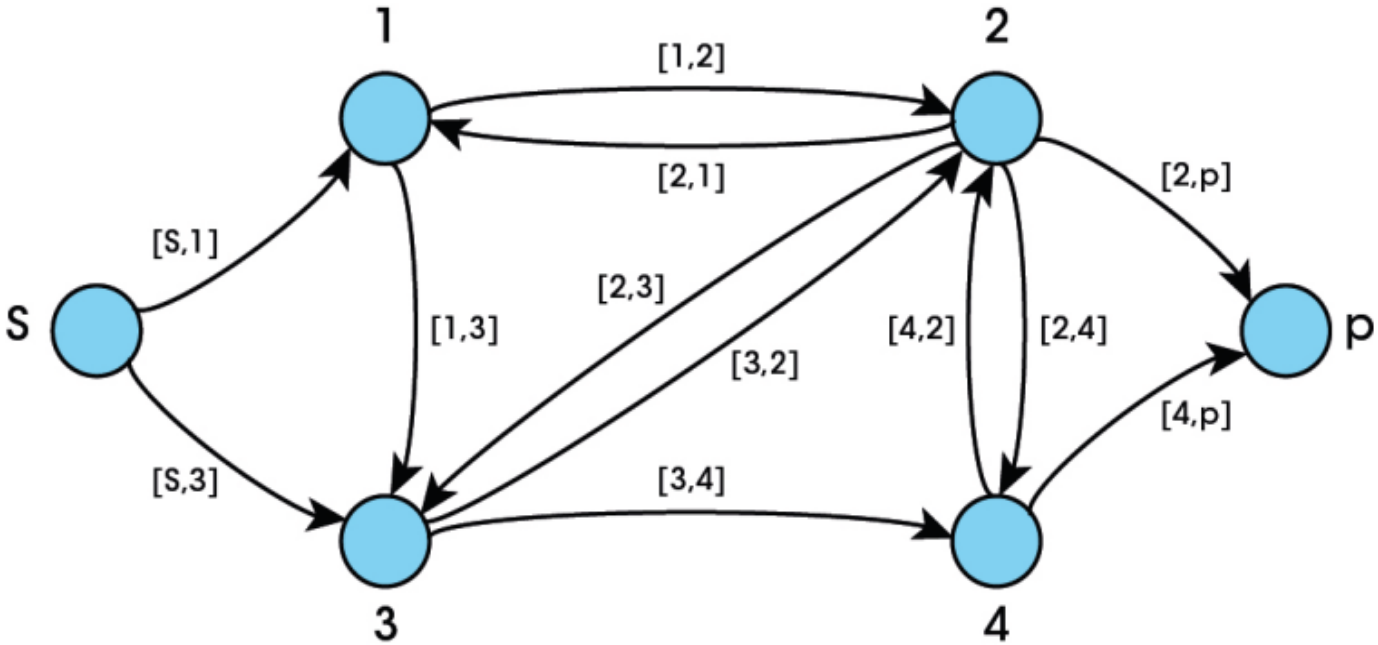
1. Programmation linéaire continue
2. Algorithme du simplexe
3. Dualité
4. Programmation linéaire discrète
   - Problèmes de flot
   - Problèmes entiers célèbres



Définition

Graphe orienté : $\mathrm{G} = (\mathcal{N}, \mathcal{A})$ *où* $\mathcal{A} \subset \mathcal{N} \times \mathcal{N}$

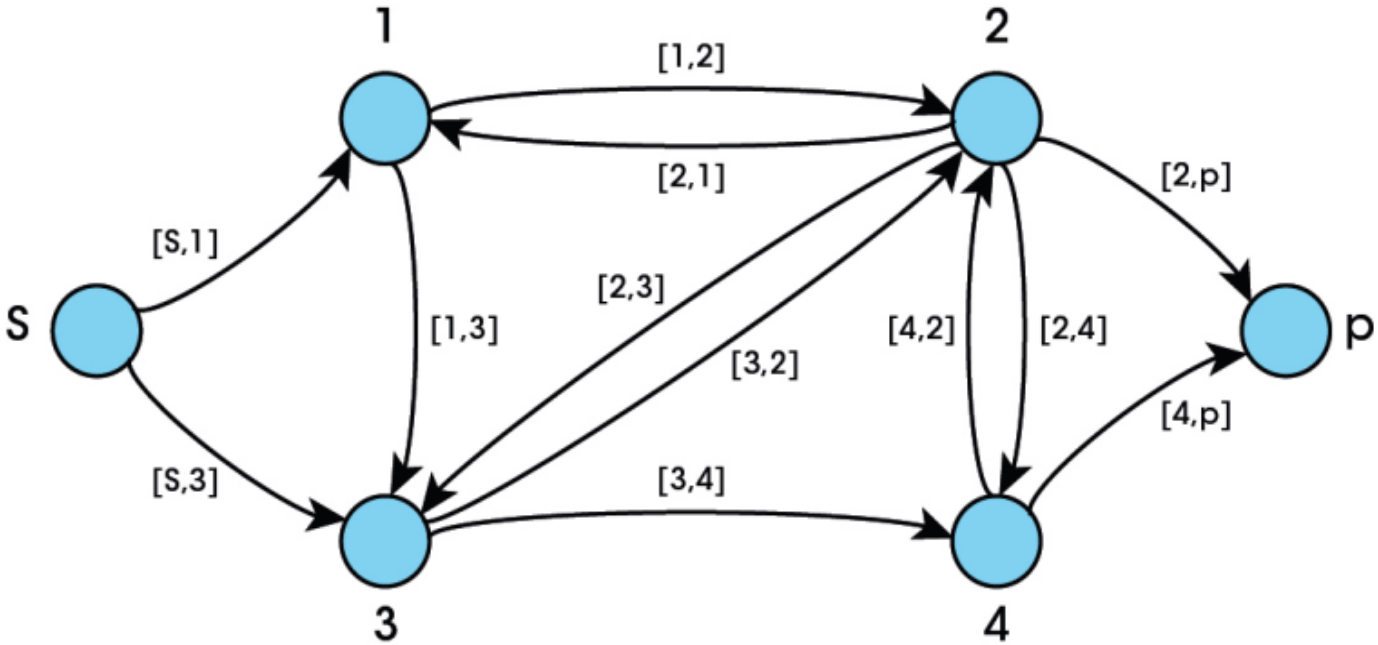
- *m noeuds et n arcs*



Définition

Graphe orienté : $G = (\mathcal{N}, \mathcal{A})$ *où* $\mathcal{A} \subset \mathcal{N} \times \mathcal{N}$

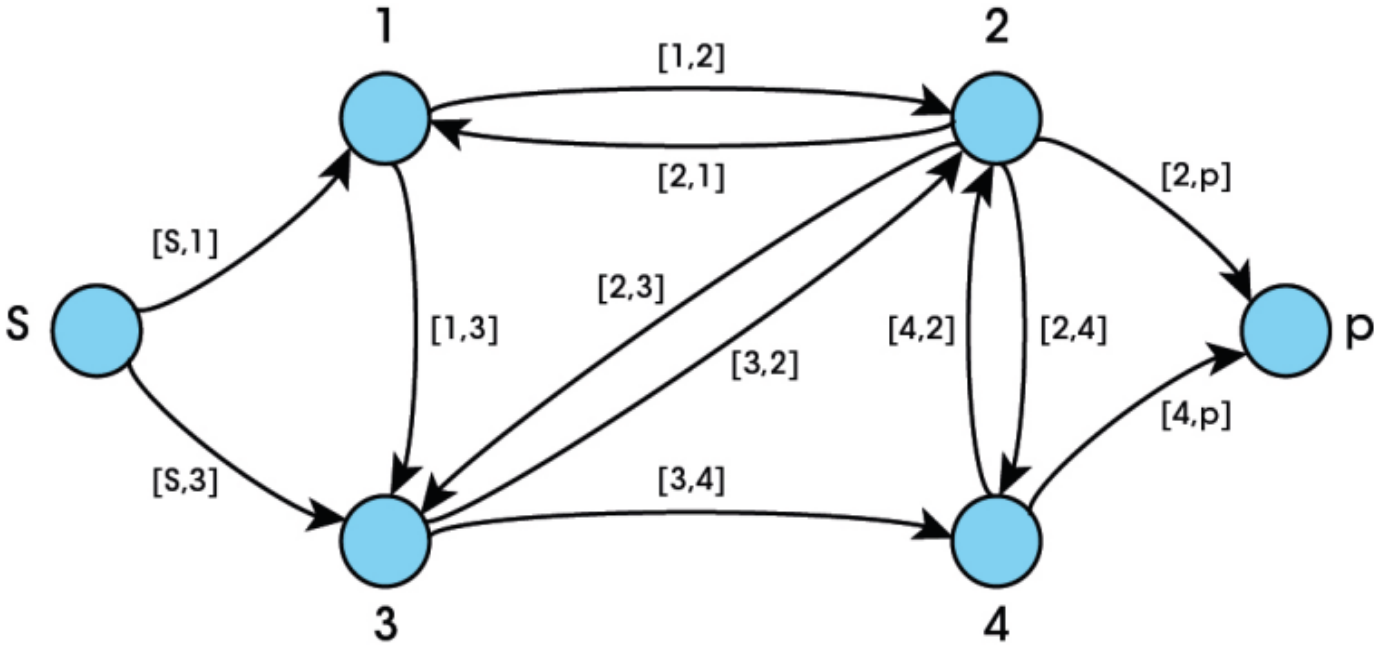
- *m noeuds et n arcs*
- $(i, j) \in \mathcal{A}$ *: arc orienté de i à j*



Définition

Graphe orienté : $G = (\mathcal{N}, \mathcal{A})$ *où* $\mathcal{A} \subset \mathcal{N} \times \mathcal{N}$

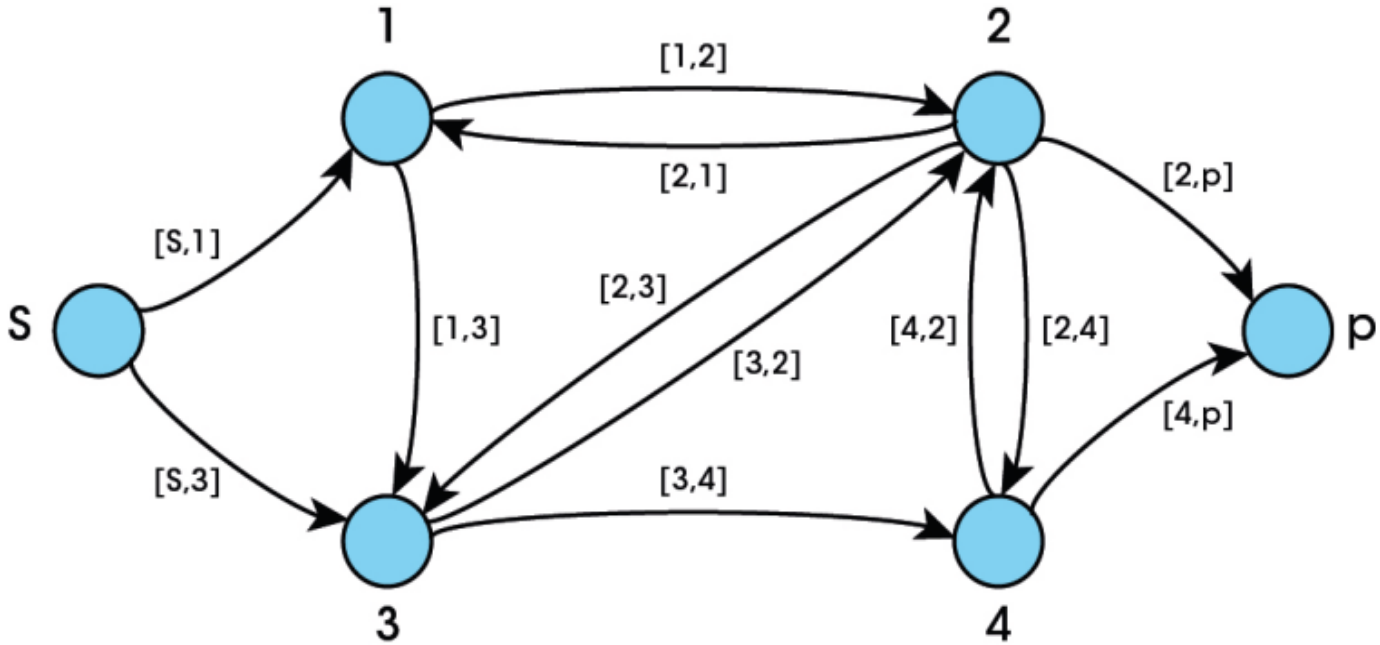
- *m noeuds et n arcs*
- $(i, j) \in \mathcal{A}$ *: arc orienté de i à j*
- *Capacité* $u_{ij} \geqslant 0$



Définition

Graphe orienté : $G = (\mathcal{N}, \mathcal{A})$ *où* $\mathcal{A} \subset \mathcal{N} \times \mathcal{N}$

- *m noeuds et n arcs*
- $(i, j) \in \mathcal{A}$ *: arc orienté de i à j*
- *Capacité* $u_{ij} \geqslant 0$
- *Coût :* c_{ij}



Définition

Graphe orienté : $G = (\mathcal{N}, \mathcal{A})$ *où* $\mathcal{A} \subset \mathcal{N} \times \mathcal{N}$

- *m noeuds et n arcs*
- $(i, j) \in \mathcal{A}$ *: arc orienté de i à j*
- *Capacité* $u_{ij} \geqslant 0$
- *Coût :* c_{ij}
- *Flot extérieur* b_i *en* i *entrant ou sortant*

Définition

On appelle flot admissible un vecteur $x \in \mathbb{R}_+^n$ *de composantes* x_{ij} *positives satisfaisant la loi des noeuds de Kirchoff :*

$$b_i + \sum_{\substack{j \in \mathcal{N} \\ (j,i) \in \mathcal{A}}} x_{ji} = \sum_{\substack{j \in \mathcal{N} \\ (i,j) \in \mathcal{A}}} x_{ij} \qquad \forall i \in \mathcal{N}$$

et vérifiant les conditions de capacité : $x_{ij} \leqslant u_{ij}$ *pour tout* $(i,j) \in \mathcal{A}$.

Définition

On appelle flot admissible un vecteur $x \in \mathbb{R}_+^n$ *de composantes* x_{ij} *positives satisfaisant la loi des noeuds de Kirchoff :*

$$b_i + \sum_{\substack{j \in \mathcal{N} \\ (j,i) \in \mathcal{A}}} x_{ji} = \sum_{\substack{j \in \mathcal{N} \\ (i,j) \in \mathcal{A}}} x_{ij} \qquad \forall i \in \mathcal{N}$$

et vérifiant les conditions de capacité : $x_{ij} \leqslant u_{ij}$ *pour tout* $(i,j) \in \mathcal{A}$.

Propriété

Soit A *la matrice de* $\mathcal{M}_{m \times n}$ *définie par* $A_{i,l} = 1$ *si* $l = (j,i) \in \mathcal{A}$, $A_{i,l} = -1$ *si* $l = (i,j) \in \mathcal{A}$ *et* $A_{i,l} = 0$ *pour tout autre arc* l *non relié à* i. *Alors*

- $Ax = b$ *(*x *réécrit comme vecteur indexé par les arcs)*

Définition

On appelle flot admissible un vecteur $x \in \mathbb{R}_+^n$ *de composantes* x_{ij} *positives satisfaisant la loi des noeuds de Kirchoff :*

$$b_i + \sum_{\substack{j \in \mathcal{N} \\ (j,i) \in \mathcal{A}}} x_{ji} = \sum_{\substack{j \in \mathcal{N} \\ (i,j) \in \mathcal{A}}} x_{ij} \qquad \forall i \in \mathcal{N}$$

et vérifiant les conditions de capacité : $x_{ij} \leqslant u_{ij}$ *pour tout* $(i,j) \in \mathcal{A}$.

Propriété

Soit A *la matrice de* $\mathcal{M}_{m \times n}$ *définie par* $A_{i,l} = 1$ *si* $l = (j,i) \in \mathcal{A}$, $A_{i,l} = -1$ *si* $l = (i,j) \in \mathcal{A}$ *et* $A_{i,l} = 0$ *pour tout autre arc* l *non relié à* i. *Alors*

- $Ax = b$ *(*x *réécrit comme vecteur indexé par les arcs)*
- A *est totalement unimodulaire.*

Définition

Un flot de coût minimum est un flot admissible minimisant le coût global $\sum_{(i,j)\in\mathcal{A}} x_{ij}c_{ij}$.

Définition

Un flot de coût minimum est un flot admissible minimisant le coût global $\sum_{(i,j)\in\mathcal{A}} x_{ij}c_{ij}$.

Applications : Affectations, emploi du temps, approvisionnement

Définition

Un flot de coût minimum est un flot admissible minimisant le coût global $\sum_{(i,j)\in\mathcal{A}} x_{ij}c_{ij}$.

Applications : Affectations, emploi du temps, approvisionnement

- Soient c, u les vecteurs de composantes c_{ij}, u_{ij}. Alors le problème de coût minimum s'écrit

$$\min_{\substack{0\leqslant x\leqslant u \\ Ax=b}} c\cdot x$$

Définition

Un flot de coût minimum est un flot admissible minimisant le coût global $\sum_{(i,j)\in\mathcal{A}} x_{ij}c_{ij}$.

Applications : Affectations, emploi du temps, approvisionnement

- Soient c, u les vecteurs de composantes c_{ij}, u_{ij}. Alors le problème de coût minimum s'écrit

$$\min_{\substack{0\leqslant x\leqslant u \\ Ax=b}} c\cdot x$$

- l'existence d'un tel flot est assurée si $c_{ij} \geqslant 0$ ou si $u_{ij} < \infty$

Définition

Un flot de coût minimum est un flot admissible minimisant le coût global $\sum_{(i,j)\in\mathcal{A}} x_{ij}c_{ij}$.

Applications : Affectations, emploi du temps, approvisionnement

- Soient c, u les vecteurs de composantes c_{ij}, u_{ij}. Alors le problème de coût minimum s'écrit

$$\min_{\substack{0\leqslant x\leqslant u \\ Ax=b}} c\cdot x$$

- l'existence d'un tel flot est assurée si $c_{ij} \geqslant 0$ ou si $u_{ij} < \infty$
- Si u et b sont entiers alors l'algorithme du simplexe fournit une solution optimale entière.

Exemple

Un constructeur automobile doit livrer son modèle AA à 2 concessionnaires à partir de 2 usines de production. Les disponibilités aux usines sont respectivement de 100 et 80 unités tandis que les demandes des vendeurs (concessionnaires) sont de 80 et 75. Les coûts c_{ij} de livraison des automobiles (de l'entrepot i vers le concessionnaire j), en centaines d'euro sont les suivants : $c_{11} = 8$, $c_{12} = 15$, $c_{21} = 10$, $c_{22} = 4$.

Résoudre ce problème comme un problème de flot à l'aide de l'agorithme du simplexe.
Aide : On introduira un concessionnaire imaginaire qui reçoit tout le stock restant avec un coût de transport nul.

Flots particuliers

Définition

Un flot maximal sur un graphe muni d'une source s (sans antécédent) et d'un puits p (sans successeur) est un flot admissible maximisant b_s où b est tel que $\mathrm{b}_\mathrm{p} = -\mathrm{b}_s$ et $\mathrm{b}_\mathrm{i} = 0$ pour $\mathrm{i} \notin \{\mathrm{s}, \mathrm{p}\}$.

Exemple : maximiser un flux sur un réseau

Flots particuliers

Définition

Un flot maximal sur un graphe muni d'une source s (sans antécédent) et d'un puits p (sans successeur) est un flot admissible maximisant b_s où b est tel que $b_p = -b_s$ et $b_i = 0$ pour $i \notin \{s, p\}$.

Remarques

- On rajoute l'arc (p, s) reliant le puits à la source et on maximise x_{ps} que l'on fait croître à partir de zéro tant qu'il existe une solution admissible

Exemple : maximiser un flux sur un réseau

Flots particuliers

Définition

Un flot maximal sur un graphe muni d'une source s (sans antécédent) et d'un puits p (sans successeur) est un flot admissible maximisant b_s où b est tel que $b_p = -b_s$ et $b_i = 0$ pour $i \notin \{s, p\}$.

Remarques

- On rajoute l'arc (p, s) reliant le puits à la source et on maximise x_{ps} que l'on fait croître à partir de zéro tant qu'il existe une solution admissible
- pour ce problème les c_{ij} n'interviennent pas.

Exemple : maximiser un flux sur un réseau

Flots particuliers

Définition

Un flot maximal sur un graphe muni d'une source s (sans antécédent) et d'un puits p (sans successeur) est un flot admissible maximisant b_s où b est tel que $b_p = -b_s$ et $b_i = 0$ pour $i \notin \{s, p\}$.

Remarques

- On rajoute l'arc (p, s) reliant le puits à la source et on maximise x_{ps} que l'on fait croître à partir de zéro tant qu'il existe une solution admissible
- pour ce problème les c_{ij} n'interviennent pas.
- $b_p = -b_s$ est nécessaire par conservation globale

Exemple : maximiser un flux sur un réseau

Définition

Un chemin de coût minimum sur un graphe muni d'une source et d'un puits est un flot de coût minimum tel que $x_{ij} \in \{0, 1\}$ *et* $b_s = -b_p = 1$ *et* $b_i = 0$ *sinon.*

Définition

Un chemin de coût minimum sur un graphe muni d'une source et d'un puits est un flot de coût minimum tel que $x_{ij} \in \{0, 1\}$ *et* $b_s = -b_p = 1$ *et* $b_i = 0$ *sinon.*

Remarques

- Les solutions optimales sont connexes car si $x_{ji} = 1$ alors $x_{ij'} = 1$ pour un certain $j' \neq j$ par admissibilité.

Définition

Un chemin de coût minimum sur un graphe muni d'une source et d'un puits est un flot de coût minimum tel que $x_{ij} \in \{0, 1\}$ *et* $b_s = -b_p = 1$ *et* $b_i = 0$ *sinon.*

Remarques

- Les solutions optimales sont connexes car si $x_{ji} = 1$ alors $x_{ij'} = 1$ pour un certain $j' \neq j$ par admissibilité.
- L'algorithme du simplexe donne la solution optimale en relaxant les contraintes : $0 \leqslant x_{ij} \leqslant 1$. Cependant l'algorithme de parcourt de graphe de Dijkstra est meilleur (complexité plus petite).

Application : itinéraire optimal sur google maps

Plan

1. Programmation linéaire continue
2. Algorithme du simplexe
3. Dualité
4. Programmation linéaire discrète
   - Problèmes de flot
   - Problèmes entiers célèbres

Problème du sac-à-dos

Soient N objets de poids p_j et d'utilité u_j. On veut prendre les objets les plus utiles sans dépasser un poids maximal L :

$$\max_{x_j \in \{0,1\},\ p \cdot x < L} u \cdot x$$

- $x \in \{0,1\}^N$: variable de décision

Problème du sac-à-dos

Soient N objets de poids p_j et d'utilité u_j. On veut prendre les objets les plus utiles sans dépasser un poids maximal L :

$$\max_{x_j \in \{0,1\},\ p \cdot x < L} u \cdot x$$

- $x \in \{0,1\}^N$: variable de décision
- Problème NP-complet (difficile). Complexité exponentielle

Problème du sac-à-dos

Soient N objets de poids p_j et d'utilité u_j. On veut prendre les objets les plus utiles sans dépasser un poids maximal L :

$$\max_{x_j \in \{0,1\},\ p \cdot x < L} u \cdot x$$

- $x \in \{0,1\}^N$: variable de décision
- Problème NP-complet (difficile). Complexité exponentielle
- On peut envisager en premier lieu une heuristique : algorithme glouton

Problème du sac-à-dos

Soient N objets de poids p_j et d'utilité u_j. On veut prendre les objets les plus utiles sans dépasser un poids maximal L :

$$\max_{x_j \in \{0,1\},\ p \cdot x < L} u \cdot x$$

- $x \in \{0,1\}^N$: variable de décision
- Problème NP-complet (difficile). Complexité exponentielle
- On peut envisager en premier lieu une heuristique : algorithme glouton
- Résolution par l'algorithme de Ford et Bellman.

Programmation dynamique. Algorithme de Ford et Bellman

Problème type

- On souhaite minimiser/maximiser $D = c_{x_0 x_1} + \ldots + c_{x_{N-1} x_N} + b_{x_N}$ qui décrit le coût/revenu total d'un processus passant par différents états x_i.
- Soit G le graphe dont les nœuds sont les x_i et les arcs les transitions possibles d'un état à un autre.
- Problème à horizon fini : minimiser/maximiser D pour $(x_j)_{1 \leqslant j \leqslant N}$ chemins de longueur N (horizon) du graphe issu de x_0. Les c_{ij} sont les coûts/rétributions sur chaque arc et b_i les coûts/rétributions sur chaque nœud.

Programmation dynamique. Algorithme de Ford et Bellman

Problème type

- On souhaite minimiser/maximiser $D = c_{x_0x_1} + \ldots + c_{x_{N-1}x_N} + b_{x_N}$ qui décrit le coût/revenu total d'un processus passant par différents états x_i.
- Soit G le graphe dont les nœuds sont les x_i et les arcs les transitions possibles d'un état à un autre.
- Problème à horizon fini : minimiser/maximiser D pour $(x_j)_{1 \leqslant j \leqslant N}$ chemins de longueur N (horizon) du graphe issu de x_0. Les c_{ij} sont les coûts/rétributions sur chaque arc et b_i les coûts/rétributions sur chaque nœud.

Principe de Bellman

Tout sous-chemin d'un chemin dans un graphe réalisant un optimum est lui aussi optimal.

Equation de Bellman à horizon fini

Définition

On appelle fonction valeur, notée $v^{\mathrm{N}} : \ x_0 \longrightarrow v^{\mathrm{N}}_{x_0}$ *où* $v^{\mathrm{N}}_{x_0}$ *est la solution du problème à horizon* N.

Equation de Bellman à horizon fini

Définition

On appelle fonction valeur, notée $v^N : x_0 \to v^N_{x_0}$ *où* $v^N_{x_0}$ *est la solution du problème à horizon* N.

Proposition

Pour le problème de minimisation la fonction valeur v^N *satisfait la récurrence*

$$v^0_x = b_x, \qquad \text{pour tout nœud } x$$

$$v^n_x = \inf_{y \in \mathcal{N}} c_{xy} + v^{n-1}_y, \quad 1 \leqslant n \leqslant N, \quad \forall\, x \in \mathcal{N}$$

Exemple

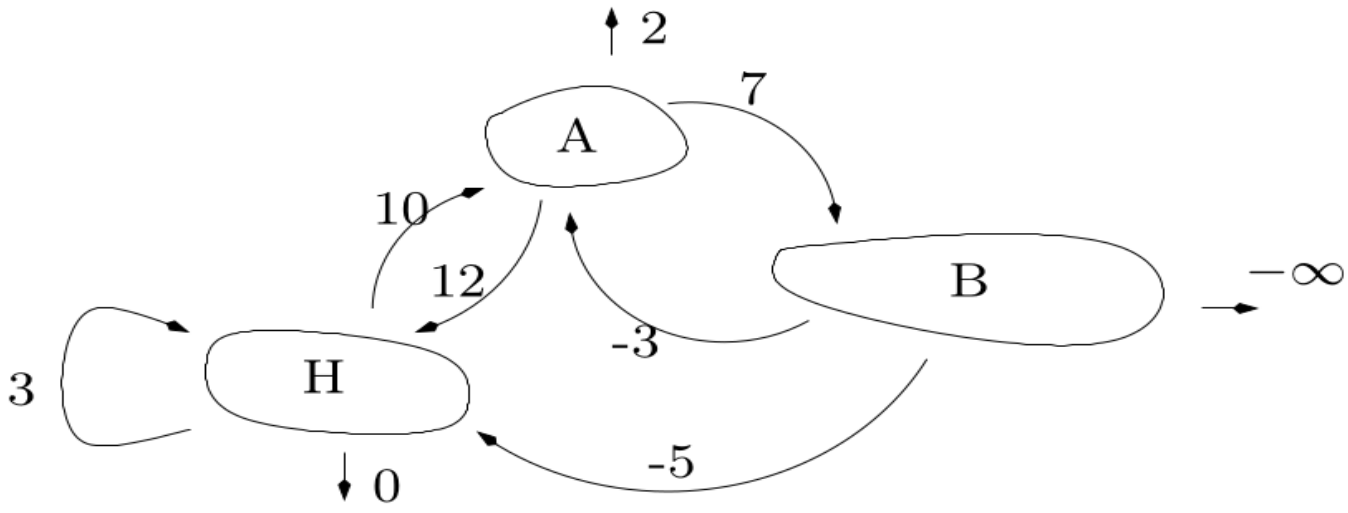


Figure – Chauffeur de taxi : H : quartier huppé ; A : aéroport ; B : banlieue

Calculer le revenu maximal du chauffeur de Taxi à horizon 2 partant de H.

Problème du sac-à-dos

$$\max_{x_j \in \{0,1\},\ p \cdot x \leqslant L} u \cdot x$$

1. Les “états” du sac-à-dos en cours de remplissage sont les divers poids $q \in [0, L]$ de celui-ci.

Problème du sac-à-dos

$$\max_{x_j \in \{0,1\},\ p \cdot x \leqslant L} u \cdot x$$

1. Les "états" du sac-à-dos en cours de remplissage sont les divers poids $q \in [0, L]$ de celui-ci.
2. La fonction valeur v^n est l'utilité d'un sac-à-dos partiel de poids inférieur à L rempli de n objets.

Problème du sac-à-dos

$$\max_{x_j \in \{0,1\},\ p \cdot x \leqslant L} u \cdot x$$

1. Les "états" du sac-à-dos en cours de remplissage sont les divers poids $q \in [0, L]$ de celui-ci.
2. La fonction valeur v^n est l'utilité d'un sac-à-dos partiel de poids inférieur à L rempli de n objets.

Equation de Bellman :

$$v_q^n = \max(v_q^{n-1}, \max_i u_i + v_{q-p_i}^{n-1}), \quad \forall\ q \leqslant L \quad \text{et } n \leqslant N.$$

Problème du représentant de commerce (travelling salesman problem)

Soient N villes de coordonnées x_n. trouver la plus courte tournée du représentant passant une seule fois par chaque ville.
Soit $\mathcal{E}_N$ l'ensemble des permutations de $\{1, \ldots, N\}$.

$$\min_{\sigma \in \mathcal{E}_N} \sum_{n=1}^{N} d(x_{\sigma(n)}, x_{\sigma(n+1)}).$$

Problème du représentant de commerce (travelling salesman problem)

Soient N villes de coordonnées x_n. trouver la plus courte tournée du représentant passant une seule fois par chaque ville.
Soit $\mathcal{E}_N$ l'ensemble des permutations de $\{1, \dots, N\}$.

$$\min_{\sigma \in \mathcal{E}_N} \sum_{n=1}^{N} d(x_{\sigma(n)}, x_{\sigma(n+1)}).$$

Reformulation : variable de décision binaire $x \in \{0,1\}^{\mathcal{A}}$.

Problème du représentant de commerce (travelling salesman problem)

Soient N villes de coordonnées x_n. trouver la plus courte tournée du représentant passant une seule fois par chaque ville.
Soit $\mathcal{E}_N$ l'ensemble des permutations de $\{1, \dots, N\}$.

$$\min_{\sigma \in \mathcal{E}_N} \sum_{n=1}^{N} d(x_{\sigma(n)}, x_{\sigma(n+1)}).$$

Reformulation : variable de décision binaire $x \in \{0,1\}^{\mathcal{A}}$.

1. chaque sommet de la tournée est connecté à 2 arrêtes : $\sum_j x_{ij} = 2$ pour tout sommet i.

Problème du représentant de commerce (travelling salesman problem)

Soient N villes de coordonnées x_n. trouver la plus courte tournée du représentant passant une seule fois par chaque ville.
Soit $\mathcal{E}_N$ l'ensemble des permutations de $\{1, \dots, N\}$.

$$\min_{\sigma \in \mathcal{E}_N} \sum_{n=1}^{N} d(x_{\sigma(n)}, x_{\sigma(n+1)}).$$

Reformulation : variable de décision binaire $x \in \{0,1\}^{\mathcal{A}}$.

1. chaque sommet de la tournée est connecté à 2 arrêtes : $\sum_j x_{ij} = 2$ pour tout sommet i.
2. connexité de la tournée complète : toute restriction de la tournée à un sous-ensemble S de noeuds n'est pas un cycle et a donc un nombre d'arrêtes strictement inférieur à card(S) : $\sum_{(i,j) \in S \cap \mathcal{A}} x_{ij} \leqslant \text{card}(S) - 1$.

Problème du représentant de commerce (travelling salesman problem)

Soient N villes de coordonnées x_n. trouver la plus courte tournée du représentant passant une seule fois par chaque ville.
Soit $\mathcal{E}_N$ l'ensemble des permutations de $\{1, \ldots, N\}$.

$$\min_{\sigma \in \mathcal{E}_N} \sum_{n=1}^{N} d(x_{\sigma(n)}, x_{\sigma(n+1)}).$$

Reformulation : variable de décision binaire $x \in \{0,1\}^{\mathcal{A}}$.

1. chaque sommet de la tournée est connecté à 2 arrêtes : $\sum_j x_{ij} = 2$ pour tout sommet i.
2. connexité de la tournée complète : toute restriction de la tournée à un sous-ensemble S de noeuds n'est pas un cycle et a donc un nombre d'arrêtes strictement inférieur à card(S) : $\sum_{(i,j) \in S \cap \mathcal{A}} x_{ij} \leqslant \text{card}(S) - 1$.
3. Problème reformulé : $\min d_{ij} x_{ij}$ sous 1 et 2.

Algorithme de Séparation, évaluation et coupe

- Description des tours sous forme d'arbre

Algorithme de Séparation, évaluation et coupe

- Description des tours sous forme d'arbre
- calcul d'une borne inférieure m initialisée à l'aide d'une heuristique particulière.

Algorithme de Séparation, évaluation et coupe

- Description des tours sous forme d'arbre
- calcul d'une borne inférieure m initialisée à l'aide d'une heuristique particulière.
- Calcul d'une borne inférieure prévisionnelle en relâchant les contraintes et en utilisant la dualité (relaxation lagrangienne)

Algorithme de Séparation, évaluation et coupe

- Description des tours sous forme d'arbre
- calcul d'une borne inférieure m initialisée à l'aide d'une heuristique particulière.
- Calcul d'une borne inférieure prévisionnelle en relâchant les contraintes et en utilisant la dualité (relaxation lagrangienne)
- Parcours de l'arbre en élagant les branches dont la borne prévisionnelle excède m.

Quelques idées pour modéliser certains liens logiques entre des variables ou des contraintes du problème. Soient a, b et c des évênements correspondant aux variables de décisions **binaires** x_a, x_b, x_c.

- Si a et b ne peuvent pas se produire tous les deux : $x_a + x_b \leqslant 1$.

Quelques idées pour modéliser certains liens logiques entre des variables ou des contraintes du problème. Soient a, b et c des évênements correspondant aux variables de décisions **binaires** x_a, x_b, x_c.

- Si a et b ne peuvent pas se produire tous les deux : $x_a + x_b \leqslant 1$.
- Si l'un au moins des évènements parmi a et b doit se produire : $x_a + x_b \geqslant 1$.

Quelques idées pour modéliser certains liens logiques entre des variables ou des contraintes du problème. Soient a, b et c des évênements correspondant aux variables de décisions **binaires** x_a, x_b, x_c.

- Si a et b ne peuvent pas se produire tous les deux : $x_a + x_b \leqslant 1$.
- Si l'un au moins des évènements parmi a et b doit se produire : $x_a + x_b \geqslant 1$.
- Si a se produit alors b doit se produire $x_a \leqslant x_b$.

Quelques idées pour modéliser certains liens logiques entre des variables ou des contraintes du problème. Soient a, b et c des évênements correspondant aux variables de décisions **binaires** x_a, x_b, x_c.

- Si a et b ne peuvent pas se produire tous les deux : $x_a + x_b \leqslant 1$.
- Si l'un au moins des évènements parmi a et b doit se produire : $x_a + x_b \geqslant 1$.
- Si a se produit alors b doit se produire $x_a \leqslant x_b$.
- On peut noter que l'inégalité précédente modélise heureusement la contraposée de la proposition logique correspondante : si b ne se produit pas, a ne doit pas se produire.

Quelques idées pour modéliser certains liens logiques entre des variables ou des contraintes du problème. Soient a, b et c des évênements correspondant aux variables de décisions **binaires** x_a, x_b, x_c.

- Si a et b ne peuvent pas se produire tous les deux : $x_a + x_b \leqslant 1$.
- Si l'un au moins des évènements parmi a et b doit se produire : $x_a + x_b \geqslant 1$.
- Si a se produit alors b doit se produire $x_a \leqslant x_b$.
- On peut noter que l'inégalité précédente modélise heureusement la contraposée de la proposition logique correspondante : si b ne se produit pas, a ne doit pas se produire.
- Si a se produit, alors b et/ou c doivent se produire : $x_a \leqslant x_b + x_c$.